ParkOps: D1 - Michael Wiszniewski - EECS 3311 - Mark Atta Mensah

This document contains an overview and explanation of the design of ParkOps. It goes over some of the decisions I made and their justifications, responsibilities of different components, two operation contracts, and a limitation. The full class diagram at the end shows my main domain and app responsibilities, visibility within the application, and associations with multiplicities. Included also is a sequence diagram which shows the process of reading B1, evaluating B12, and creating a pending proposal.

Valid states and how each class protects them

Class	**Constraint (valid state requirements)**	**How it is enforced by the code**
Identifier	{value matches [A-Z][A-Z0-9_-]{0,15}}	A single rule. isValid(String) checks it without throwing for the request boundary. The constructor throws an IllegalArgumentException for invalid identifiers on the same rule, so every Identifier that exists is valid. It is immutable and compared by value.
Duration	{1 <= hours <= 24}	The rule is shared by isValid(int) and the constructor. It is compared by value and immutable. DomainRules only delegates to the Identifier and Duration classes. Therefore each rule lives in one place.
Booking	{id, spaceId, duration not null; version >= 0}	The constructor rejects any invalid input. All fields are private and final with getters and no setter. IDs and duration are of the type Identifier and Duration respectively, so they are already valid (see above rows).
Space	{id not null}	The constructor rejects a null ID by throwing an IllegalArgumentException. The three booleans cannot be invalid. Fields are private and final with getters only.
Proposal	{all fields not null; bookingVersion >= 0; policyVersion >= 1}	The constructor rejects invalid input by throwing an IllegalArgumentException. The object is immutable and only stores the three IDs, the booking and policy versions, and a ProposalStatus.

Requirement checklist

Requirement	**Where the design meets it**
Validate identifiers and duration	The value objects Identifier and Duration Malformed request text becomes a Refused result (TARGET_MALFORMED or BOOKING_MALFORMED) before any object is built.
Proposed reassignment is not equal to completed change	A Proposal does not have a reference to a Booking or Space object. Every field is both private and final, and Proposal has getters only no setters. ProposalManager never changes a booking or space, because the only state it writes is a PENDING proposal saved in ProposalStore. The outcome types are sealed because Created always holds a proposal, and Refused has no proposal field. Therefore a proposed reassignment is not the same as a completed change.
Separate object instances	Every Booking, Space and Proposal object is its own instance, keyed by its Identifier. ParkingRegistry and ProposalStore reject any duplicate IDs (see putIfAbsent method). SequentialIdGenerator gives each new proposal a fresh ID (enumerated as PROP_1, PROP_2, and so on…). As seen in the sequence diagram, B1 is unchanged after PROP_1 is created.

Requirement	Where the design meets it
Controlled state access with no mutable internal collection exposed	To ensure controlled state access, there are only getters, no setters. ParkingRegistry exposes only findSpace and findBooking, which return Optional, and has no mutator. The bookingSnapshot() method returns an unmodifiable Map.of(...), rather than the Booking object itself. The internal Maps are private and never returned. Using List.copyOf, the Constructor input is copied. findByBooking returns List.copyOf(...) which copies the collection structure. Elements are immutable so the copy cannot be used for hidden writes. The only write path is ProposalStore.save, which has the role of validating.

Relationship choices

- ParkingRegistry is associated with Space/Booking ("Knows-a relationship"), it does not create them. The registry receives objects that already exist through its constructor, and only copies the collection. Having them as a field does not make it an exclusive owner.
- ProposalStore is associated with Proposal ("Knows-a relationship").
- No object link from Booking or Proposal to Space in the implementation view. In order to get from Booking/Proposal to Space, the ID is looked up in the Parking registry. The Java code holds spaceId, bookingId and targetId as Identifier values. The conceptual links are visible in the class diagram, where the dashed line ID reference links show those multiplicities.
- Realization relationship ("Can-do") instead of generalization ("Is-a"). Each of PolicyV1, SequentialIdGenerator, Created/Refused and Eligible/Ineligible implement interfaces, drawn with a dashed line and a hollow triangle. No interface is drawn as a superclass.
- Every dependency is labelled with their meaning (for example: creates, returns). Value types and enumerations appear as attribute types instead of adding extra lines.

Responsibilities of the application classes

ProposalManager	When a request comes in, it runs it through the checks in order to decide if a proposal is eligible or not. It stores eligible proposals as pending and does not modify any booking or space.
ReassignmentPolicy/PolicyV1	Evaluates a target's eligibility for a booking based on the policy, and reports the policy version to allow the proposal to know which policy evaluated it. These classes do not perform lookups or create proposals.
ParkingRegistry	Holds each space and booking. Can lookup spaces and bookings by Identifier, which returns an Optional. It does not have a mutator.
ProposalStore	Holds all proposals of any status from the moment they are created. It can both store and get proposals, and find proposals by bookingId. It only returns immutable or unmodifiable copies of proposals
IdGenerator / SequentialIdGenerator	Creates proposal IDs (PROP_1, PROP_2, etc…). For the purpose of testing, this is the only place they are created.
StudentApplication	Serves as an adapter to the given starter API. It takes the supplied fixture and builds the parking registry from it. It calls the ProposalManager, and maps Created to ProposalView, and throws an IllegalArgumentException on Refused.

Rejected alternative: My design has two outcome types for proposal requests: Created and Refused. Created outcomes hold a Proposal, and on Refused requests no proposal is created. A Refused outcome only holds a reason code and a message, it can't hold a Proposal at all because the field doesn't exist. An alternative to this approach was having one Result class with a status code with an Optional<Proposal> field that either holds a Proposal or is empty. For a refusal, the box is empty, and no Proposal object exists anywhere. The drawback to this alternative is that an Optional would still allow a refused status to carry a Proposal, and it would allow a success status with an empty box. Both of these scenarios contradict the logic of the Optional<Proposal> box, yet nothing in the type prevents it. This would have to be enforced in the constructor, or it will not be caught by the compiler. The Optional version is simpler to implement, and the separate types version has the cost of added complexity (more types), and every caller needs to switch over both outcomes in the studentApplication.propose method, but I liked that the separate types version makes the two invalid states described unrepresentable, so I chose the separate types version.

Two informal operation contracts:

1. ProposalManager.proposeReassignment is used to assign proposal requests to the Created or Refused type.

Precondition: None, it will accept any string and never throw an exception.

Postconditions per outcome:

- If the target ID is malformed, it returns as Refused with code TARGET_MALFORMED.
- Otherwise if the booking ID is malformed, it returns as Refused with code BOOKING_MALFORMED.
- Otherwise if either isn't in the registry, it returns as Refused with BOOKING_UNKNOWN, then TARGET_UNKNOWN.
- Otherwise if a policy rule fails, it returns as Refused with CURRENT_SPACE, then OCCUPIED, then NOT_OPEN (whichever policy rule fails first).
- Otherwise it returns as Created, which holds a PENDING proposal. The proposal has the ID from the supplied IdGenerator, the booking ID, the target ID, the booking's version (at the time of the call), and the policy's version.

State changes when the result is Created: ProposalStore gains one new Proposal with the status of PENDING, and the IdGenerator that was supplied to the ProposalManager is incremented by one.

State changes when the result is Refused: no changes at all.

What remains unchanged: Regardless of the result, the ParkingRegistry, current bookings, and existing proposals are not modified throughout this process.

2. Duration.isValid is used to validate Durations for the Duration constructor.

Precondition: None.

Post condition per outcome:

- The inputted duration is between 1 and 24 inclusively: returns true.
- Otherwise: returns false

State changes: since isValid is static, it does not change any state. It receives an int, and never throws.

Most important limitation: The system does not currently have a measure of handling duplicate proposals. Submitting the exact same proposal twice would create two PENDING proposals for the same booking, version and target.

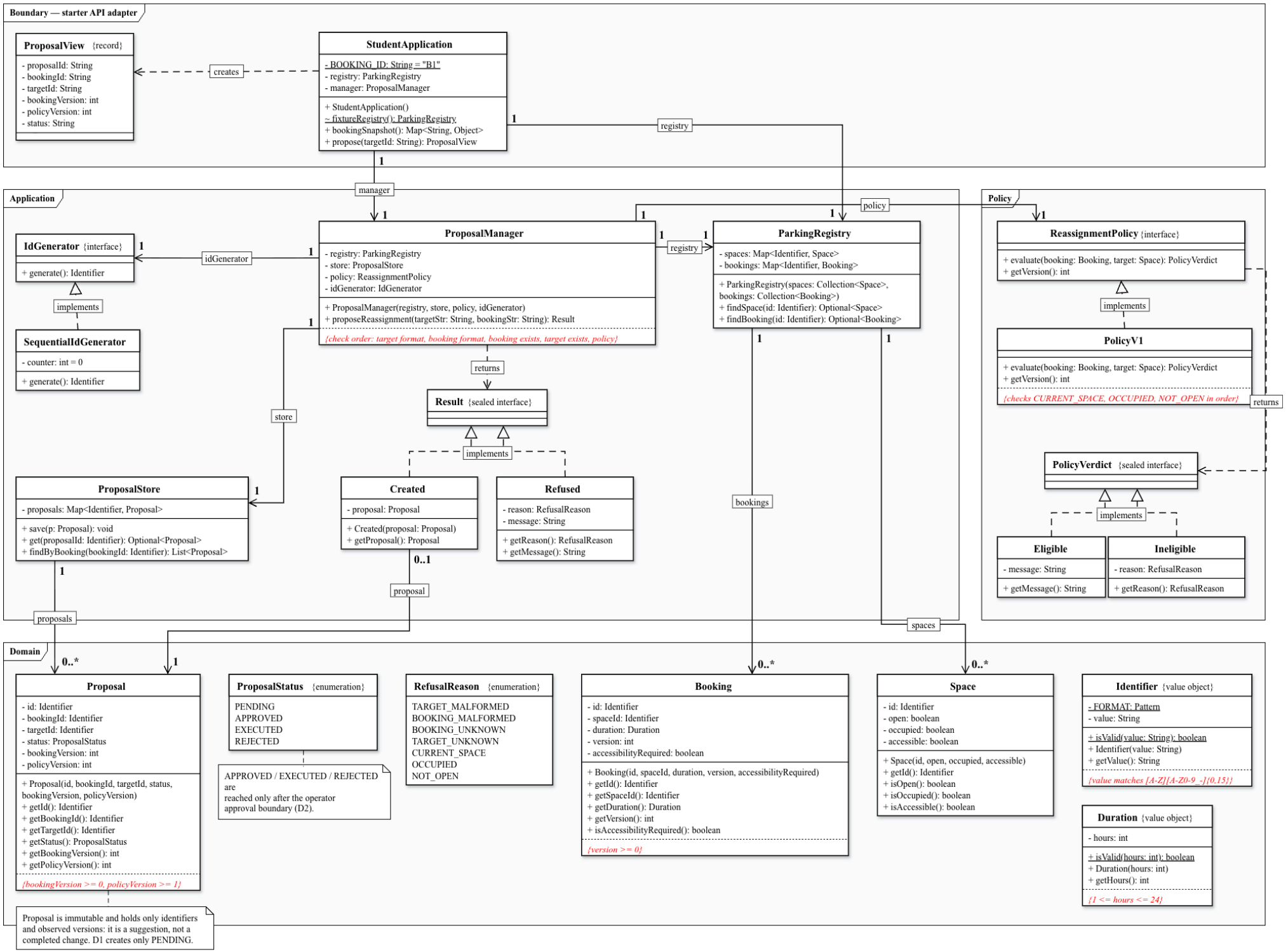
What infrastructure was reused, and which parts I implemented

Reused and unchanged	ChatMessage, Fixture, Main, MiniJson, ModelProvider, OllamaProvider, ProposalView, ReplayProvider Scripts: run.sh, run.ps1, setup-junit.sh, setup-junit.ps1. Checks: test/plain/InfrastructureChecks.java, test/plain/PublicChecks.java, test/junit/InfrastructureTest.java, test/junit/SuppliedChecksTest.java. Data: data/scenario.json, data/replay.jsonl. Notes: START_HERE.md, STARTER_PROVENANCE.md, results.txt,
Reused and modified	DomainRules: requireDuration and requireIdentifier now delegate to Duration.isValid and Identifier.isValid. StudentApplication: builds the parking registry from Fixture (package-private fixtureRegistry()), and propose(String) calls ProposalManager.proposeReassignment, which maps Created to ProposalView and Refused to the IllegalArgumentException. bookingSnapshot() now reads from the registry, and run(...) is a stub for D2 StudentTests.java, README.md, TEST_PLAN.md, CHANGELOG.md, AI_USE.md
I wrote	Identifier, Duration, ProposalStatus, RefusalReason, Space, Booking, Proposal, ParkingRegistry, ProposalStore, IdGenerator, SequentialIdGenerator, ReassignmentPolicy, PolicyV1, PolicyVerdict, Eligible, Ineligible, Result, Created, Refused, ProposalManager, and the tests in StudentTests

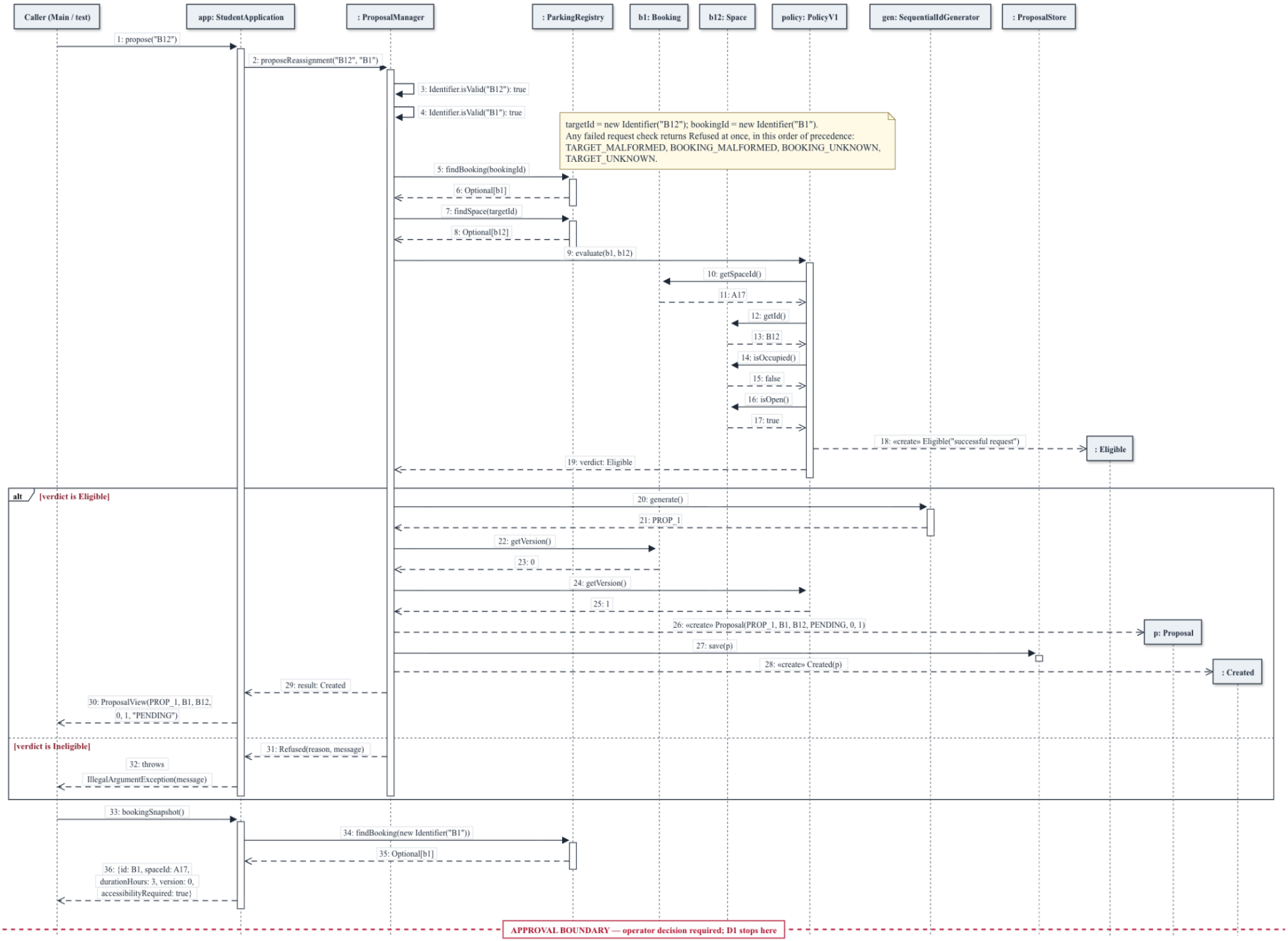
Notation explanations

Solid line with open arrowhead, name of the role and the two multiplicities	Navigable association with the multiplicity nearest a class showing how many of that class one object at the other end is linked to.
Dashed line with hollow triangle, labelled implements	Realization of an Interface.
Dashed line with open arrowhead and a label	Dependency where the type is used without a lasting link.
+, -, ~, underlined	Public, private, package visibility, underline means static.
{...} in red italics	A constraint that is enforced in the code.
Sequence: solid line with filled arrowhead / dashed line with open arrowhead	Synchronous call / reply. A dashed <<create>> message ends at the new object's head. **alt** is a combined fragment whose operands have guards.

ParkOps D1 — Class diagram (implementation view: domain and application responsibilities)



ParkOps D1 — Sequence diagram: read B1, evaluate B12, create a PENDING proposal



No message crosses this line in D1. PROP_1 stays PENDING; B1 is still in A17 at version 0 and B12 is still unoccupied.
An operator approve / reject / execute (with rechecks of approval, versions and eligibility) belongs after this boundary (D2).